GA7-220501096-AA4-EV03 Componente front-end del proyecto formativo y proyectos de clase (listas de chequeo)

Ficha:3070324

Programa: análisis y desarrollo de software

juan Santiago Arango puentes

Servicio nacional de aprendizaje Sena

Instructor: FRANCISCO ARNALDO VARGAS

2025

Evidencia de Producto: GA7-220501096-AA4-EV03

Componente frontend del proyecto formativo y proyectos de clase (listas de chequeo)

1. Introducción

Esta evidencia documenta el desarrollo del componente frontend del proyecto Menu Pizza, una aplicación web construida con React JS y Vite. Está elaborada siguiendo los lineamientos del componente "Desarrollo de frontend con React JS" y los requisitos del estándar SENA para la evidencia GA7-220501096-AA4-EV03.

2. Objetivo

Demostrar la codificación del módulo frontend del proyecto formativo utilizando buenas prácticas, comentarios en el código, estándares de codificación y control de versiones. La aplicación debe responder a un sistema web real con componentes reutilizables, manejo de estado y navegación entre pantallas.

3. Alcance del proyecto

- Plataforma: Web
- Framework: React JS
- Herramienta de construcción: Vite
- Lenguaje: TypeScript
- Control de versiones: Git y GitHub
- Artefactos previos considerados: diagrama de clases, diagramas de casos de uso, historias de usuario, diseño de interfaz, prototipos y plan de trabajo técnico.

4. Artefactos del ciclo de software considerados

1. Diagrama de clases
   - Describe las entidades del dominio: Usuario, Producto, Carrito y Administrador.
2. Diagrama de casos de uso
   - Acciones principales: Iniciar sesión, ver menú, agregar al carrito, gestionar pedidos, administrar usuarios.
3. Historias de usuario
   - Como cliente quiero explorar pizzas para elegir y pedir rápidamente.
   - Como administrador quiero gestionar usuarios y ver el historial de pedidos.
4. Diseño y prototipos
   - Se contempló una interfaz clara, navegable, con botones accesibles y estados visuales.
5. Plan de trabajo técnico
   - Definición de tecnología: React JS, TypeScript, Vite, Tailwind CSS.
   - Versionamiento: inicio del repositorio local y remoto en GitHub.

5. Descripción del módulo frontend

El módulo frontend está compuesto por varios componentes principales:

- `App.tsx` : controla las vistas y el flujo principal de la aplicación.
- `Home.tsx` : pantalla de bienvenida con opciones de navegación.
- `Menu.tsx` : presenta productos con opciones de añadir al carrito.
- `ShoppingCart.tsx` : muestra el contenido del carrito y permite actualizar cantidades.

- `Login.tsx` y `Register.tsx` : gestionan autenticación básica.
- `PaymentMethods.tsx` y `LocationForm.tsx` : gestionan medios de pago y dirección.
- `AdminPanel.tsx` y `UserManagement.tsx` : componentes de administración.

6. Estándares de codificación aplicados

- Nombres de archivos en PascalCase para componentes React.
- Uso de `interface` y `type` en TypeScript para tipado estático.
- Separación entre presentación y lógica con componentes pequeños y claros.
- Comentarios descriptivos en funciones y secciones críticas.
- Uso de `const` para valores inmutables y `let` solo cuando es necesario.
- Estructura de carpetas coherente: `components` , `assets` , `utils` .

7. Control de versiones

El proyecto fue inicializado con Git y sincronizado con GitHub. El repositorio remoto registrado es:

- `https://github.com/juantronixpro-bit/MENU-PIZZA.git`

Se usó el flujo básico:

- `git init`
- `git add .`
- `git commit -m "Initial commit for Menu Pizza project"`
- `git remote add origin <url>`
- `git push -u origin master`

8. Lista de chequeo de componentes frontend

Item	Descripción	Cumplimiento
Estructura de proyecto	Componentes organizados en carpetas lógicas	Sí
Uso de React	Componentes funcionales y Hooks	Sí
Tipado TypeScript	Interfaces y tipos definidos	Sí
Comentarios en el código	Se incluyen explicaciones en bloques importantes	Sí
Control de versiones	Repositorio local y remoto configurado	Sí
Diseño responsive	Componentes preparados para layout móvil/web	Sí
Buenas prácticas	Nombres descriptivos y separación de responsabilidades	Sí
Pruebas básicas	Construcción exitosa con `npm run build`	Sí

9. Código de ejemplo con comentarios

```
// App.tsx
import { useState, useEffect } from 'react';
import { Home } from './components/Home';
import { Menu } from './components/Menu';
import { ShoppingCart } from './components/ShoppingCart';
import { PaymentMethods } from './components/PaymentMethods';
import { Login } from './components/Login';
```

```typescript
import { Register } from './components/Register';
import { LocationForm } from './components/LocationForm';
import { AdminPanel } from './components/admin/AdminPanel';
import { UserManagement } from './components/admin/UserManagement';

export type View =
  | 'home'
  | 'menu'
  | 'cart'
  | 'login'
  | 'register'
  | 'location'
  | 'payment'
  | 'admin';

export interface Product {
  id: string;
  name: string;
  price: number;
  size: string;
}

export function App() {
  // Estado global de la aplicación: vista activa y carrito
  const [view, setView] = useState<View>('home');
  const [cart, setCart] = useState<Product[]>([]);

  const addToCart = (product: Product) => {
    // Agrega el producto seleccionado al carrito
    setCart((current) => [...current, product]);
  };

  return (
    <main>
      {view === 'home' && <Home onNavigate={setView} />}
      {view === 'menu' && (
        <Menu onAddToCart={addToCart} onViewCart={() => setView('cart')} />
      )}
      {view === 'cart' && <ShoppingCart cart={cart} onBack={() => setView('menu')} />}
    </main>
  );
}
```

10. Procedimiento de validación

1. Validar el correcto tipado en TypeScript.
2. Ejecutar `npm install` para instalar dependencias.
3. Ejecutar `npm run build` para confirmar que no hay errores de compilación.
4. Abrir el proyecto en `http://localhost:5173` en modo desarrollo.
5. Revisar el estado visual de las vistas y la navegación entre componentes.

Diagrama de despliegue:

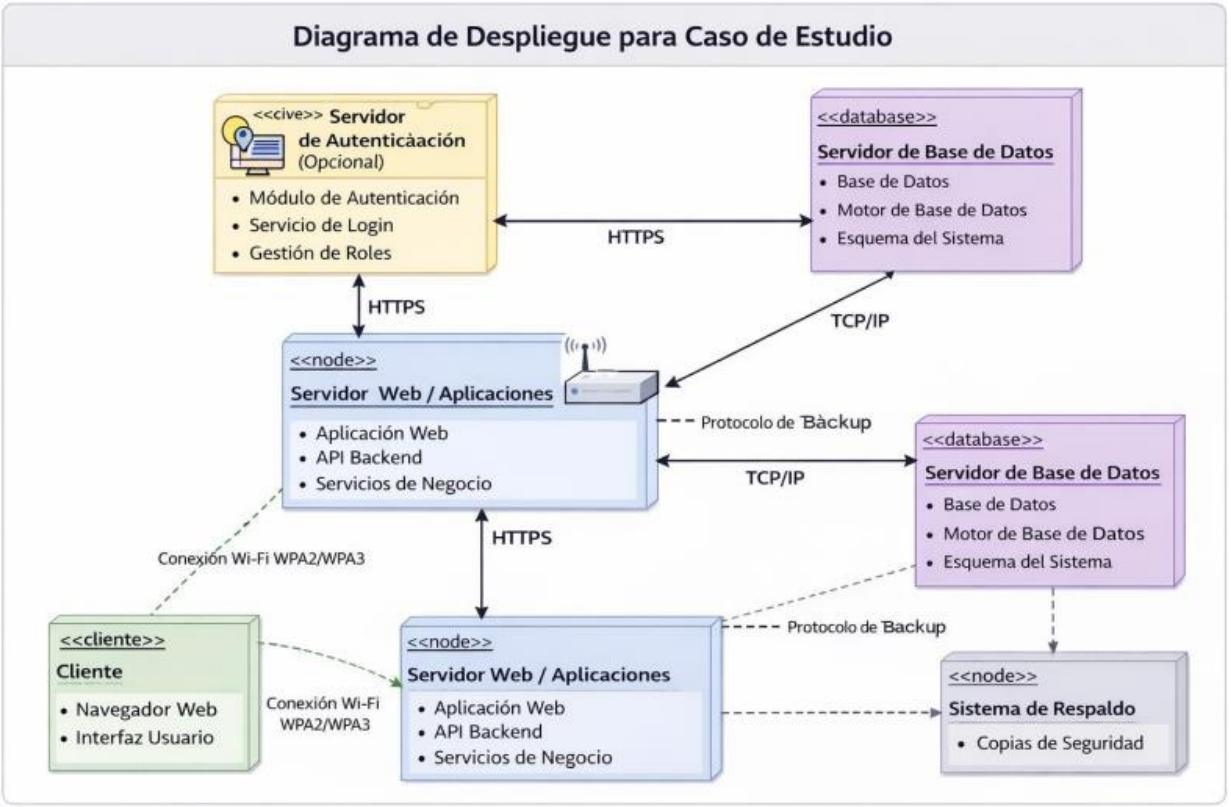
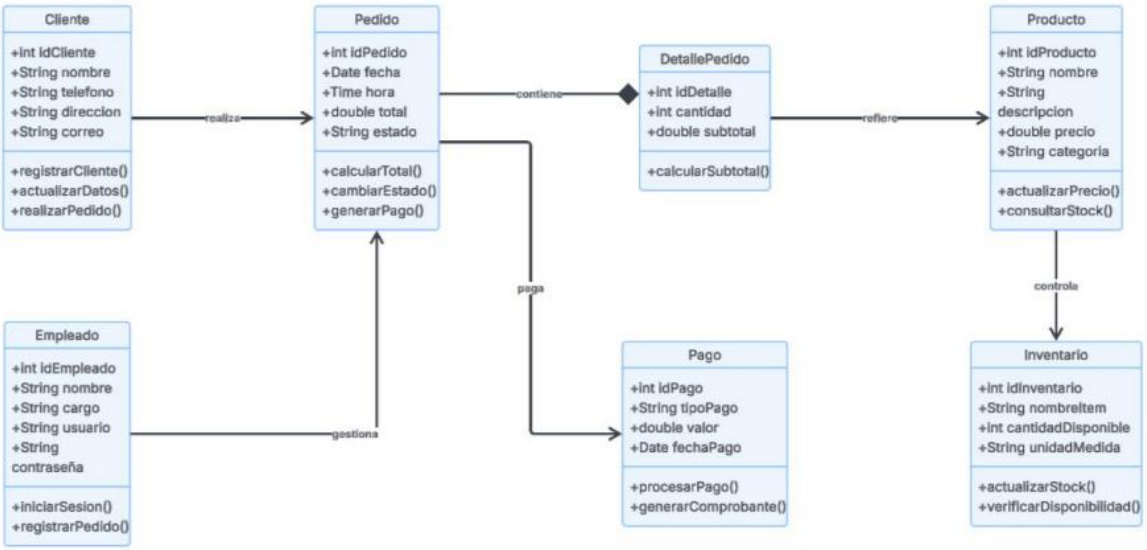


Diagrama de clases:



Casos de uso:

CU01 – Iniciar Sesión

Campo	Descripción
Nombre del Caso de Uso	Iniciar Sesión
Actor Principal	Cliente y Administrador
Actores Secundarios	Ninguno

Precondiciones	El usuario debe tener una cuenta previamente registrada. El sistema debe estar disponible y accesible.
Flujo Principal	1. El usuario accede a la página principal. 2. Selecciona la opción "Iniciar Sesión". 3. Ingresa su email o usuario (ADMIN1, ADMIN2, ADMIN3) y contraseña. 4. El sistema valida las credenciales. 5. El sistema crea una sesión activa y asigna el rol correspondiente. 6. El usuario es redirigido a su área: Menú (Cliente) o Panel Administrativo (Administrador).
Flujos Alternativos	3a. Credenciales incorrectas → Se muestra mensaje de error y permite reintentar (máximo 3 intentos). 3b. Usuario olvida contraseña → Opción de recuperación de contraseña.
Excepciones	Usuario bloqueado por intentos fallidos → Se muestra mensaje y bloqueo temporal.
Postcondiciones	El usuario queda autenticado en el sistema y la sesión permanece activa.
Relaciones	<<include>> Validar Credenciales
Requisitos Asociados	RF02 – Inicio de sesión

CU02 – Registrar Nuevo Usuario

Campo	Descripción
Nombre del Caso de Uso	Registrar Nuevo Usuario
Actor Principal	Cliente
Precondiciones	El usuario no tiene cuenta registrada previamente.
Flujo Principal	1. El usuario selecciona "Crear Cuenta". 2. Ingresa nombre completo, email, teléfono, contraseña y confirma contraseña. 3. El sistema valida que el email no exista. 4. El sistema guarda los datos del nuevo usuario. 5. Muestra mensaje de éxito y redirige a Iniciar Sesión.
Flujos Alternativos	3a. Email ya registrado → Mensaje de error. 3b. Contraseñas no coinciden → Mensaje de error y limpiar campos.
Postcondiciones	Se crea una nueva cuenta de cliente en el sistema.
Relaciones	<<include>> Validar Datos de Registro
Requisitos Asociados	RF01 – Registro de usuario

CU03 – Visualizar Menú y Seleccionar Tamaño

Campo	Descripción
Nombre del Caso de Uso	Visualizar Menú y Seleccionar Tamaño
Actor Principal	Cliente
Precondiciones	El cliente ha accedido al sistema.
Flujo Principal	1. El cliente selecciona "Ver Menú". 2. El sistema muestra todas las pizzas con imagen, nombre, descripción y precios. 3. El cliente selecciona una pizza. 4. El sistema muestra las opciones de tamaño (Pequeña, Mediana, Grande). 5. El cliente selecciona el tamaño deseado. 6. El sistema actualiza el precio dinámicamente.

Postcondiciones	La pizza queda lista para ser agregada al carrito con el tamaño y precio correcto.
Relaciones	<<include>> Seleccionar Tamaño de Pizza
Requisitos Asociados	RF04, RF05

CU04 – Agregar Producto al Carrito

Campo	Descripción
Nombre del Caso de Uso	Agregar Producto al Carrito
Actor Principal	Cliente
Precondiciones	El cliente ha seleccionado una pizza y su tamaño.
Flujo Principal	1. El cliente presiona "Agregar al Carrito". 2. El sistema agrega el producto con tamaño y precio correspondiente. 3. Actualiza el contador del carrito en tiempo real. 4. Muestra mensaje de confirmación.
Postcondiciones	El producto queda agregado al carrito del usuario.
Relaciones	<<include>> Seleccionar Tamaño de Pizza
Requisitos Asociados	RF06

CU05 – Registrar Ubicación de Entrega

Campo	Descripción
Nombre del Caso de Uso	Registrar Ubicación de Entrega
Actor Principal	Cliente
Precondiciones	El cliente tiene productos en el carrito y desea finalizar el pedido.
Flujo Principal	1. El sistema obliga al cliente a ingresar ubicación antes del pago. 2. El cliente ingresa ciudad, dirección completa y datos adicionales (opcional). 3. El sistema valida los campos obligatorios. 4. Los datos se guardan temporalmente para el pedido actual.
Postcondiciones	La ubicación queda registrada y se habilita el paso siguiente (método de pago).
Relaciones	<<include>> desde Confirmar Pedido
Requisitos Asociados	RF10

CU06 – Realizar Pago

Campo	Descripción

Nombre del Caso de Uso	Realizar Pago
Actor Principal	Cliente
Precondiciones	El carrito tiene productos, ubicación registrada y método de pago seleccionado.
Flujo Principal	1. El cliente revisa el resumen del pedido (subtotal, IVA, total). 2. Selecciona el método de pago. 3. Confirma el pago. 4. El sistema registra el pedido completo.
Postcondiciones	El pedido queda confirmado, el carrito se vacía y se genera número de pedido.
Relaciones	<<include>> Registrar Ubicación de Entrega <<include>> Seleccionar Método de Pago
Requisitos Asociados	RF11, RF12

CU07 – Gestionar Productos (Administrador)

Campo	Descripción
Nombre del Caso de Uso	Gestionar Productos
Actor Principal	Administrador
Precondiciones	El administrador ha iniciado sesión con rol ADMIN.
Flujo Principal	1. El administrador accede al Panel Administrativo. 2. Selecciona "Gestión de Productos". 3. Puede Crear, Editar o Eliminar pizzas (nombre, descripción, precios por tamaño, imagen). 4. Los cambios se reflejan inmediatamente en el menú visible para clientes.
Postcondiciones	El catálogo de productos queda actualizado.
Relaciones	<<extend>> Crear Producto <<extend>> Editar Producto <<extend>> Eliminar Producto
Requisitos Asociados	RF14

CU08 – Gestionar Usuarios (Administrador)

Campo	Descripción
Nombre del Caso de Uso	Gestionar Usuarios
Actor Principal	Administrador
Precondiciones	El administrador ha iniciado sesión.
Flujo Principal	1. Accede a "Gestión de Usuarios". 2. Puede ver lista de usuarios, banear, eliminar o reactivar cuentas.
Postcondiciones	El estado de los usuarios queda actualizado.
Requisitos Asociados	RF13

CU09 – Consultar Historial de Pedidos

Campo	Descripción
Nombre del Caso de Uso	Consultar Historial de Pedidos
Actor Principal	Cliente y Administrador
Precondiciones	El usuario ha iniciado sesión.
Flujo Principal	1. El usuario accede a su historial. 2. El sistema muestra lista de pedidos con fecha, productos, total y estado.
Postcondiciones	El usuario puede ver el detalle de sus pedidos.
Requisitos Asociados	RF17

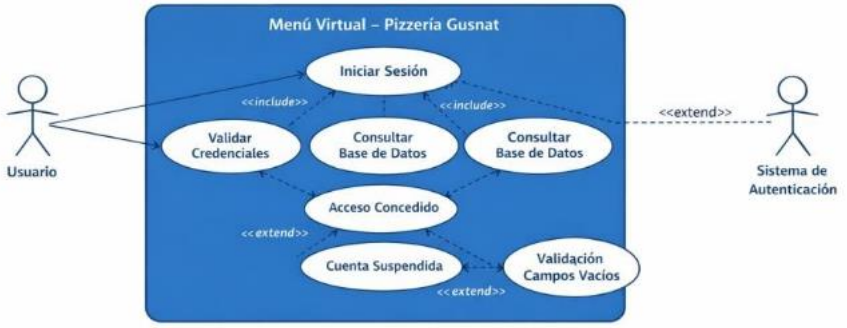
CU10 – Planificar en Calendario Administrativo

Campo	Descripción
Nombre del Caso de Uso	Planificar en Calendario Administrativo
Actor Principal	Administrador
Precondiciones	El administrador ha iniciado sesión.
Flujo Principal	1. Accede al Calendario Administrativo. 2. Puede programar cambios en menú, promociones o suspensiones. 3. Asigna responsables y fechas.
Postcondiciones	Los eventos quedan programados en el calendario.
Requisitos Asociados	RF19

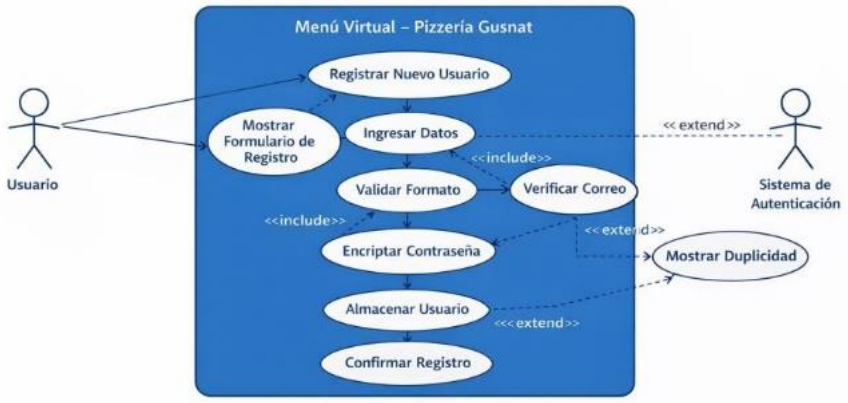
REQUISITOS NO FUNCIONALES

Código	Requisito	Descripción
RNF01	Usabilidad	La interfaz debe ser intuitiva, consistente y fácil de usar, siguiendo la paleta de colores negro y naranja (#F97513).
RNF02	Seguridad	Control de acceso por roles (Cliente y Administrador). Validación de credenciales y protección de sesión.
RNF03	Persistencia	El carrito de compras y la sesión del usuario deben mantenerse en localStorage.
RNF04	Rendimiento	Tiempo de carga de páginas inferior a 2 segundos. Actualizaciones en tiempo real del carrito y precios.
RNF05	Responsividad	El sistema debe ser completamente adaptable a dispositivos móviles, tablets y escritorio (mobile-first).
RNF06	Accesibilidad	Cumplir con contraste mínimo 4.5:1, navegación por teclado y etiquetas ARIA.
RNF07	Mantenibilidad	Código modular, organizado en componentes y con tipado estricto en TypeScript.
RNF08	Compatibilidad	Funcionar correctamente en los navegadores modernos (Chrome, Firefox, Edge, Safari).

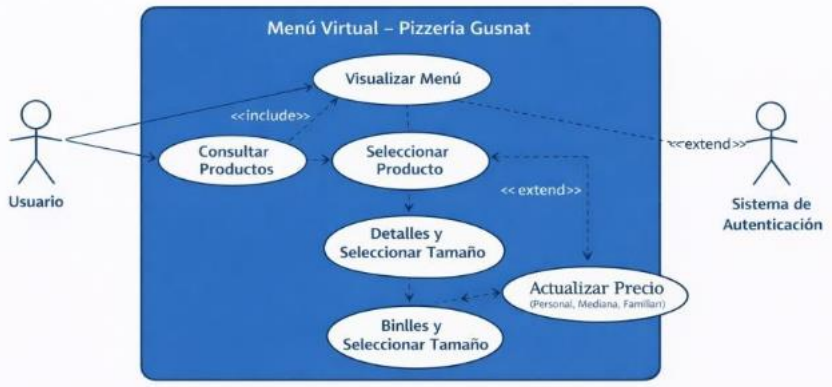
CU01 – Iniciar Sesión



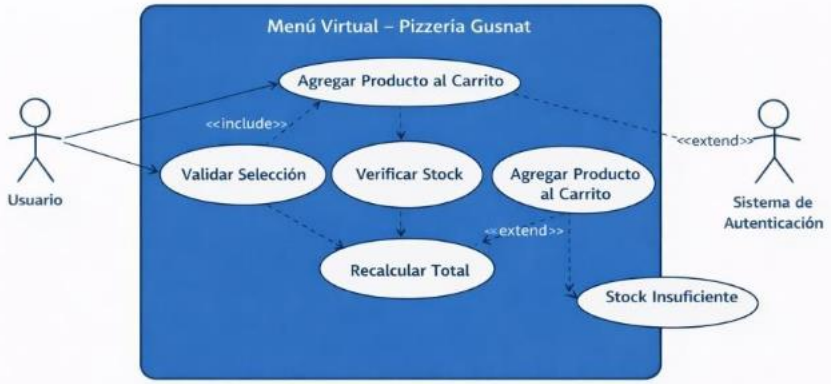
CU02 – Registrar Nuevo Usuario



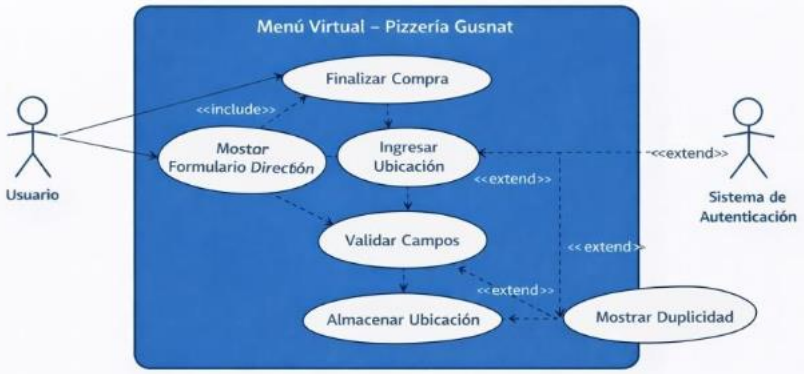
CU03 – Visualizar Menú y Seleccionar Tamaño



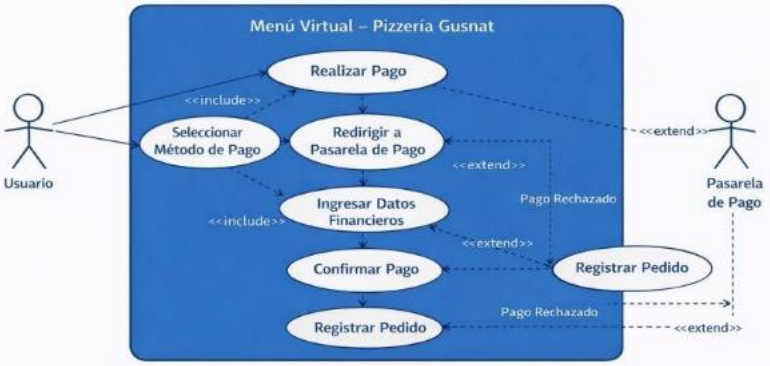
CU04 – Agregar Producto al Carrito

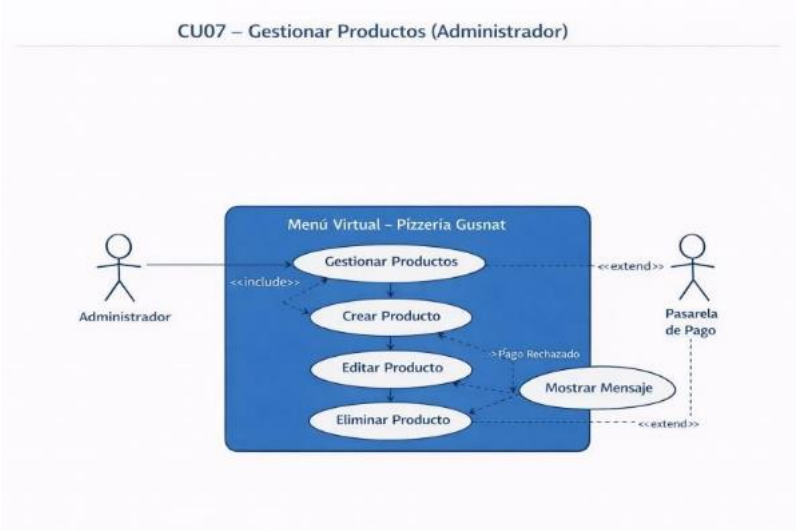


CU05 – Registrar Ubicación de Entrega



CU06 – Realizar Pago





11. Conclusión

La evidencia presentada cumple con los requisitos de codificación de frontend usando React JS. Se aplicaron los artefactos de ingeniería de software previos y se documentó el uso de control de versiones. El proyecto es una aplicación web funcional que respeta estándares de limpieza y modularidad.

12. Archivos incluidos

- Documento de evidencia en formato Markdown
- Documento de evidencia en formato PDF
- Archivo con enlace del repositorio GitHub
- Carpeta comprimida en ZIP y RAR con archivos de entrega